

Closira Inbound Call Handling Platform

System Architecture, Conversational Quality Assurance, and Handover Guide

Executive Summary

This document provides a highly detailed system blueprint and operational manual for the Closira AI Voice Agent MVP and its proposed migration to an enterprise technology stack. The voice agent autonomously manages 100% of inbound calls on behalf of small and medium businesses (SMBs), answering within 3 rings, executing custom SOPs, qualifying leads, and transferring to human representatives when necessary.

Currently deployed in the Breakout Escape Rooms test environment, the platform operates on a hybrid FastAPI/Flask stack using static schema-based memory serialization. All 838 unit tests run successfully, validating conversational recovery protocols, 14-point QA evaluation scorecards, and live monitoring dashboards. The following pages detail our conversational design, scoring layer, legacy architecture, and the cross-team integration roadmap.

1. Conversational Design & Heuristic Recovery

To maximize customer conversion rates and avoid the sterile, loop-prone tone of standard IVR systems, the agent's dialog is engineered with emotional cushions, contextual mirroring, and dynamic steering rules.

Key Conversational Paradigms:

- **Warm Inbound Opener:** Assumes positive booking intent immediately and frames the greeting around the event category (e.g., friends gathering, birthday, or corporate team event) instead of a simple dry greeting.
- **Sequence Restructuring:** Qualifies the customer's escape room experience level (*first escape room or experienced players*) prior to standard parameters, permitting tailored room suggestions.
- **Guided Qualification Mirroring:** Rather than standard form filling, the agent contextually validates customer details. For example, if a customer says 'Bengaluru', the agent responds: *'Awesome, Bangalore is a fantastic spot. We have some really exciting rooms there.'*
- **Checkout Mirroring Summary:** Before asking for the caller's first name or phone number, the agent summarizes the locked booking parameters (e.g., date, slot time, number of players) to establish trust and confirmation.
- **Contradictory Field Resolution:** If a caller updates an answer mid-dialogue (e.g., changing group size from 8 to 12), the state updater automatically updates the variable and mirrors the adjustment.

2. Quality Assurance (QA) Scorer & 14-Point Rubric

To persistently monitor and guarantee conversational quality across sessions, a turn-by-turn evaluation layer runs concurrently inside the learning agent pipeline. The agent evaluates its own performance across 7 metrics, scoring 0 to 2 points per metric (total 14 points, quality benchmark set at ≥ 12).

Metric Category	Target Behavior rated (Score 0, 1, or 2)
1. Intent Identified	Verifies whether booking, FAQ, or complaint intents were successfully mapped.
2. Empathy Cushion	Rates emotional cushioning and objection cushions in response to pricing or location complaints.
3. Direct Answer	Ensures the agent answers policy questions directly instead of using robotic evasion templates.
4. Context Retained	Checks that the agent did not repeat questions for details already saved in memory.
5. Naturalness	Assesses response brevity (targeted under 45 words) and natural non-robotic flow.
6. Flow Advancement	Ensures the agent ends with exactly one active question or clear call to action.
7. Policy Compliance	Verifies compliance with event-routing, safety, and payment instructions.

Retroactive Hydration: All 137 historical simulation sessions located in the staging directory have been retroactively evaluated under this rubric, establishing a realistic baseline for analytics tracking.

3. Legacy MVP Architecture Flow

The initial MVP architecture provides direct integration with telephony and webhook streaming, caching session data to file directories.

Current Call Loop Lifecycle:

1. **Call Ingestion:** TeleCMI captures the inbound call and routes audio WebRTC/SIP streams to the Vapi telephony gateway.
2. **Webhook Dispatch:** Vapi triggers a POST request containing conversational text to app.py on port 8010.
3. **Agent Orchestration:** main.py coordinates the Inbound, Lead Qualification, and Booking agents, storing details in local JSON files.
4. **Quality Evaluation:** The Scoring and Learning agents calculate turn scores, appending evaluation records to the conversation memory.
5. **Monitoring Admin Portal:** The Flask app on port 8020 parses the files to compile lead lists, conversion funnels, follow-ups, and Chart.js graphs.

Payment Lock Policy:

A critical business logic constraint dictates that selecting a calendar slot does NOT constitute a booking confirmation. Instead, slots are held temporarily (pending). Only upon receiving a successful payment webhook callback is the booking state changed to CONFIRMED and a WhatsApp confirmation sent.

4. Proposed Enterprise Architecture Migration

To support production scale, high concurrency, and multi-platform access, the backend architecture is proposed to migrate to a highly decoupled enterprise stack.

Proposed Tech Stack Components:

- **Front-End:** React Native (Cross-platform mobile CRM app) with NativeWind, and React JS + Tailwind CSS for the web admin dashboard.
- **Backend Gateway:** FastAPI async web services handling Vapi tool requests and REST API calls.
- **Agentic Flow:** LangGraph for stateful agent charts with LangChain wrappers to easily swap between OpenAI, Claude, and Gemini.
- **Background Workers:** Celery tasks running on Redis brokers to handle out-of-band notifications, post-call summary generation, and safe-hours checking.
- **Real-time Streaming:** Centrifugo WebSocket server to push live transcripts and telemetry directly to the frontends without polling.
- **Databases:** PostgreSQL for structured transactional data (leads, bookings), and MongoDB for unstructured logs and transcript collections.
- **RAG Stack:** Qdrant vector database using text-embedding-3-large embeddings for semantic SOP policy search.

5. Cross-Team Integration & Alignment Roadmap

For a successful integration, the following registers and action items must be tracked and aligned across the AI core team, backend team, and infrastructure/DevOps team.

RACI Responsibility Matrix:

System Module	AI Core	Backend	DevOps	QA / Product
LangGraph State Charts	A / R	C	I	C
FastAPI Webhooks & REST	R	A / R	C	I
Postgres / MongoDB Storage	C	A / R	R	I
Qdrant Policy Vectors	A / R	C	R	C
WhatsApp & Payment APIs	C	A / R	I	R

Critical Downstream Blockers:

- WhatsApp API Access:** Meta Business credentials and approved template payloads must be finalized before WhatsApp confirmation tests can run.
- Payment Webhooks:** Cryptographic signature verification schemas must be mapped to validate transaction states safely.
- TeleCMI SIP Transfers:** Active phone group forwarding numbers must be set up to test agent takeovers.